RAG Learning

2.4 Boolean Logic

OCR GCSE Computer Science (J277) — Paper 2

Boolean logic underpins all decision-making in computers. Every condition in every program, every circuit in every processor, evaluates to either **True (1)** or **False (0)**. Logic gates are the physical electronic components that implement Boolean operations in hardware. Understanding Boolean logic is essential for both programming and understanding how computer hardware works at the fundamental level.

2.4.1 Boolean Operators and Truth Tables

The Three Core Gates: AND, OR, NOT

These three gates are the foundation. Every other gate (NAND, NOR, XOR) can be built from combinations of these three.

AND Gate

Output is TRUE only if **BOTH** inputs are TRUE. Think of two switches wired in **series** — the light only turns on if both switches are closed.

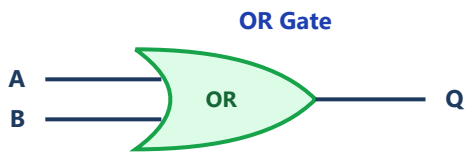
A	B	A AND B
0	0	0
0	1	0

1	0	0
1	1	**1**

Programming example: `if age >= 18 AND hasTicket == True then allowEntry()` — both conditions must be true for entry.

OR Gate

Output is TRUE if **AT LEAST ONE** input is TRUE. Think of two switches wired in **parallel** — the light turns on if either switch is closed.

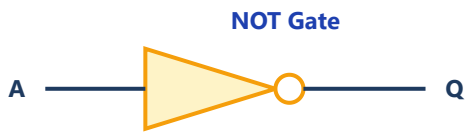


A	B	A OR B
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

Programming example: `if isStudent OR isChild then applyDiscount()` — either condition is sufficient.

NOT Gate (Inverter)

Inverts the single input. TRUE becomes FALSE, FALSE becomes TRUE. It has only one input (unlike AND and OR which have two).



A	NOT A
0	**1**
1	0

Programming example: `if NOT loggedIn then showLoginPage()` — inverts the condition.

Additional Gates: NAND, NOR, XOR

Gate	Expression	Rule	Key Fact
NAND	`NOT (A AND B)`	Opposite of AND. Output is FALSE only when **both** inputs are TRUE.	NAND is a **universal gate** — any other gate can be built from NAND gates alone. This makes it extremely important in chip design because manufacturers only need to produce one type of gate.
NOR	`NOT (A OR B)`	Opposite of OR. Output is TRUE only when **both** inputs are FALSE.	NOR is also a universal gate. Early computers were built entirely from NOR gates.
XOR	`A XOR B`	Exclusive OR. Output is TRUE only when inputs are **different**.	Used in binary addition (half adder), parity checking, and simple encryption (XOR cipher). True when "one or the other, but not both."

Complete Truth Tables for All Gates

A	B	AND	OR	NAND	NOR	XOR
0	**0**	0	0	1	1	0
0	**1**	0	1	1	0	1
1	**0**	0	1	1	0	1

© RAG Learning · Not for redistribution

1	1	1	1	0	0	0

Memory aids:

- **AND** — only 1 when ALL inputs are 1 (the strict gate)
- **OR** — only 0 when ALL inputs are 0 (the generous gate)
- **NOT** — always flips (the rebel)
- **NAND** — the opposite of AND (flip AND's output column)
- **NOR** — the opposite of OR (flip OR's output column)
- **XOR** — 1 only when inputs DIFFER (the "odd one out" gate)

Connecting Gates to Programming

```
// Every IF statement uses Boolean logic:

// AND in code:
if temperature > 20 AND weather == "sunny" then
    print("Perfect for a picnic")
endif

// OR in code:
if day == "Saturday" OR day == "Sunday" then
    print("Weekend!")
endif

// NOT in code:
if NOT fileExists("data.txt") then
    print("Error: file not found")
endif

// Combined logic:
if (isAdmin OR isModerator) AND isActive then
    showControlPanel()
endif
// Brackets matter! Without them, AND binds first:
// isAdmin OR (isModerator AND isActive) — different meaning!
```

Exam Tip: Make sure you can quickly recall the truth tables. A useful shortcut: **AND** outputs 1 in only ONE row (both inputs 1). **OR** outputs 0 in only ONE row (both inputs 0). **NAND** and **NOR** are simply the inverted outputs of AND and OR respectively. **XOR** outputs 1 when inputs are different.

2.4.2 Boolean Expressions and Constructing Truth Tables

How to Construct a Truth Table

Follow this systematic approach for any Boolean expression:

1. **List all input variables** as column headers on the left.
2. **Calculate the number of rows:** 2^n rows for n inputs. (2 inputs = 4 rows; 3 inputs = 8 rows; 4 inputs = 16 rows.)
3. **Fill in all input combinations** systematically. For 2 inputs: 00, 01, 10, 11. For 3 inputs: 000, 001, 010, 011, 100, 101, 110, 111. (Count up in binary.)
4. **Add intermediate columns** for sub-expressions (e.g., NOT B, A AND B). Work these out before the final output. This is crucial for avoiding mistakes and earning method marks.
5. **Calculate the final output** column using the intermediate results.

Worked Example 1: A AND (NOT B) — Two Inputs

Step 1: Two inputs (A, B), so 2^2 = 4 rows. Step 2: Add intermediate column for NOT B. Step 3: Calculate final output.

A	B	NOT B	A AND (NOT B)
0	**0**	1	0
0	**1**	0	0
1	**0**	1	**1**
1	**1**	0	0

Interpretation: The output is 1 only when A is true AND B is false. In code: "if the alarm is set AND the door is NOT locked, trigger the alert."

Worked Example 2: (A OR B) AND (NOT C) — Three Inputs

Three inputs = 2^3 = 8 rows. Two intermediate columns needed: (A OR B) and (NOT C).

A	B	C	A OR B	NOT C	(A OR B) AND (NOT C)
0	**0**	0	0	1	0
0	**0**	1	0	0	0
0	**1**	0	1	1	**1**
0	**1**	1	1	0	0
1	**0**	0	1	1	**1**
1	**0**	1	1	0	0
1	**1**	0	1	1	**1**
1	**1**	1	1	0	0

Interpretation: Output is 1 when at least one of A or B is true, AND C is false. In code: "if (the student has a pass OR the teacher approves) AND the exam is NOT cancelled, allow entry to the exam room."

Worked Example 3: NOT (A AND B) OR C — Three Inputs

This one requires careful attention to the order of operations. NOT applies to (A AND B) first, then the result is OR'd with C.

A	B	C	A AND B	NOT (A AND B)	NOT(A AND B) OR C
0	**0**	0	0	1	**1**

0	**0**	1	0	1	**1**
0	**1**	0	0	1	**1**
0	**1**	1	0	1	**1**
1	**0**	0	0	1	**1**
1	**0**	1	0	1	**1**
1	**1**	0	1	0	0
1	**1**	1	1	0	**1**

The output is 0 only when A=1, B=1, and C=0 (the only row where NAND gives 0 and C does not rescue it with OR).

Common Boolean Expressions in Programming

Expression	Meaning	When It Is True
`x > 5 AND x < 10`	x is between 5 and 10 (exclusive)	x = 6, 7, 8, or 9
`age < 18 OR age > 65`	Age is below 18 or above 65	A child or a senior citizen
`NOT loggedIn`	The user is NOT logged in	loggedIn is False
`score >= 50 AND attempts <= 3`	Score at least 50 with 3 or fewer attempts	Both conditions must hold simultaneously
`NOT (a == b)`	a and b are NOT equal	Equivalent to `a != b`
`(isAdmin OR isModerator) AND isActive`	User has elevated privileges AND is active	Active admin or active moderator

Writing a Boolean Expression from a Truth Table

Sometimes an exam question gives you a truth table and asks you to write the Boolean expression. The method is:

1. Find all rows where the output is **1**.
2. For each of these rows, write an AND expression using the input values (use NOT for any input that is 0).
3. OR all these AND expressions together.

```
Example: Given this truth table, find the expression:
  A  B  Q
  0  0  0
  0  1  1    ← Q=1 when A=0, B=1:  (NOT A) AND B
  1  0  1    ← Q=1 when A=1, B=0:  A AND (NOT B)
  1  1  0

Expression: Q = ((NOT A) AND B) OR (A AND (NOT B))
This is actually XOR! Q = A XOR B
```

Exam Tip: **AND** needs BOTH inputs to be True. **OR** needs at least one. **NOT** flips the value. **NAND** and **NOR** are AND/OR with the output inverted. **XOR** is True only when inputs DIFFER. When constructing truth tables, **always add intermediate columns** for sub-expressions — this avoids mistakes and earns method marks even if the final answer has an error. For 2 inputs: 4 rows. For 3 inputs: 8 rows. Count up in binary to fill in the input columns systematically (never skip or repeat a combination).

2.4.3 De Morgan's Laws

De Morgan's Laws are two fundamental rules for simplifying Boolean expressions involving NOT applied to AND or OR. They allow you to rewrite a NAND as a NOR (and vice versa), which is useful both in Boolean algebra and in logic circuit design.

De Morgan's Laws state:

Law	Expression	In Words	Equivalent Gate
First Law	`NOT (A AND B) = (NOT A) OR (NOT B)`	The negation of a conjunction is the disjunction of the negations	NAND = NOT-A OR NOT-B
Second Law	`NOT (A OR B) = (NOT A) AND (NOT B)`	The negation of a disjunction is the conjunction of the negations	NOR = NOT-A AND NOT-B

Verifying De Morgan's First Law with a Truth Table

Both sides of the equation must produce the same output for all input combinations:

A	B	A AND B	NOT(A AND B)	NOT A	NOT B	(NOT A) OR (NOT B)
0	**0**	0	**1**	1	1	**1**
0	**1**	0	**1**	1	0	**1**
1	**0**	0	**1**	0	1	**1**
1	**1**	1	**0**	0	0	**0**

The two highlighted columns are **identical** for all four input combinations — confirming that `NOT(A AND B) = (NOT A) OR (NOT B)`.

Verifying De Morgan's Second Law with a Truth Table

A	B	A OR B	NOT(A OR B)	NOT A	NOT B	(NOT A) AND (NOT B)
0	**0**	0	**1**	1	1	**1**
0	**1**	1	**0**	1	0	**0**
1	**0**	1	**0**	0	1	**0**
1	**1**	1	**0**	0	0	**0**

© RAG Learning · Not for redistribution

Using De Morgan's Laws to Simplify

A key application is rewriting expressions to use fewer or simpler gate types:

```
Example: Simplify NOT(A AND B) AND NOT(C OR D)

Step 1: Apply De Morgan's First Law to NOT(A AND B):
        = (NOT A OR NOT B) AND NOT(C OR D)

Step 2: Apply De Morgan's Second Law to NOT(C OR D):
        = (NOT A OR NOT B) AND (NOT C AND NOT D)

This eliminates the NAND and NOR gates, replacing them with
AND/OR gates plus NOT gates — which may be more practical depending
on available components.

Another example: Simplify NOT(NOT A AND NOT B)
Step 1: Apply De Morgan's First Law (note: the "AND" inside has both inputs already NOTe
        NOT(NOT A AND NOT B) = NOT(NOT A) OR NOT(NOT B)
Step 2: Double negation: NOT(NOT X) = X
        = A OR B
So NOT(NOT A AND NOT B) = A OR B — NAND gates with inverted inputs equal an OR gate!
```

Memory Aid: "Break the bar, change the operator"

```
When moving NOT (the "bar" in overline notation) inside brackets:
  1. NOT(A AND B)  →  NOT A  OR  NOT B     (AND becomes OR)
  2. NOT(A OR B)   →  NOT A  AND  NOT B    (OR becomes AND)

The rule: "break the NOT, change the AND/OR to its opposite."
AND ↔ OR, NOT distributes to each variable.
```

Exam Tip: De Morgan's Laws appear on the OCR J277 spec and are examined directly. You must be able to: (1) state both laws; (2) verify them with a truth table; (3) use them to simplify an expression. The key phrase in the mark scheme is "NOT(A AND B) is equivalent to NOT A OR NOT B" and vice versa. A very common exam question is to "use De Morgan's Law to simplify NOT(A OR B)" — the answer is "(NOT A) AND (NOT B)". Do not confuse the two laws: NOT of AND gives OR; NOT of OR gives AND.

2.4.4 Boolean Algebra Simplification

Boolean algebra is a set of rules for simplifying Boolean expressions. A simpler expression needs fewer logic gates, which means cheaper, faster, and more energy-efficient circuits. The **OCR J277 spec** requires you to apply these laws to simplify expressions.

Core Boolean Algebra Laws

Law	AND form	OR form	Meaning
Identity	`A AND 1 = A`	`A OR 0 = A`	AND with 1 / OR with 0 leaves A unchanged
Null / Annihilator	`A AND 0 = 0`	`A OR 1 = 1`	AND with 0 always gives 0; OR with 1 always gives 1
Idempotent	`A AND A = A`	`A OR A = A`	Repeating the same variable has no effect
Complement	`A AND (NOT A) = 0`	`A OR (NOT A) = 1`	A variable AND its complement is always 0; OR is always 1
Double negation	`NOT(NOT A) = A`		Two NOTs cancel each other out
Commutative	`A AND B = B AND A`	`A OR B = B OR A`	Order of inputs does not matter
Associative	`(A AND B) AND C = A AND (B AND C)`	`(A OR B) OR C = A OR (B OR C)`	Grouping does not matter for the same operator
Distributive	`A AND (B OR C) = (A AND B) OR (A AND C)`	`A OR (B AND C) = (A OR B) AND (A OR C)`	AND distributes over OR, and OR distributes over AND

Absorption	`A AND (A OR B) = A`	`A OR (A AND B) = A`	A "absorbs" the more complex expression
De Morgan's	`NOT(A AND B) = NOT A OR NOT B`	`NOT(A OR B) = NOT A AND NOT B`	Break the NOT, change the operator

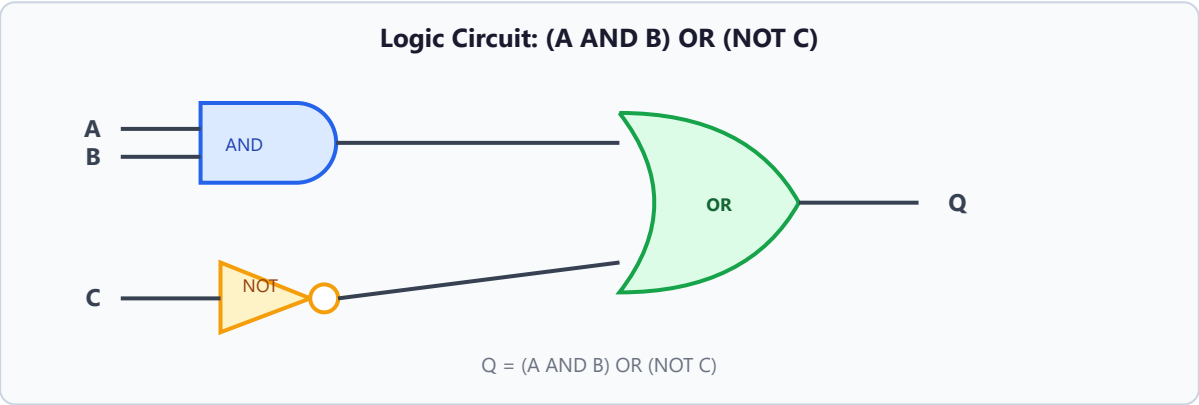
Worked Simplification Examples

```
Example 1: Simplify A AND (A OR B)

  A AND (A OR B)
= A                      [Absorption law: A AND (A OR B) = A]

Verify: if A=0, expression = 0 AND (0 OR B) = 0 AND B = 0 ✓ (matches A=0)
        if A=1, expression = 1 AND (1 OR B) = 1 AND 1   = 1 ✓ (matches A=1)
──────────────────────────────────────────────────────────

Example 2: Simplify (A AND B) OR (A AND NOT B)

  (A AND B) OR (A AND NOT B)
= A AND (B OR NOT B)          [Distributive: factor out A]
= A AND 1                     [Complement: B OR NOT B = 1]
= A                           [Identity: A AND 1 = A]

Meaning: A AND B, or A AND NOT-B — in both cases A must be true;
         whether B is true or false is irrelevant. So the output is just A.
──────────────────────────────────────────────────────────

Example 3: Simplify NOT(NOT A OR NOT B)

  NOT(NOT A OR NOT B)
= NOT(NOT A) AND NOT(NOT B)   [De Morgan's Second Law: break NOT, change OR to AND]
= A AND B                     [Double negation: NOT(NOT X) = X]

So NOT(NOT A OR NOT B) = A AND B.
A NOR gate with both inputs inverted is equivalent to an AND gate!
──────────────────────────────────────────────────────────

Example 4: Simplify A OR (A AND B) OR (NOT A AND B)

Step 1: A OR (A AND B) = A                  [Absorption]
```

```
Step 2: A OR (NOT A AND B)
      = (A OR NOT A) AND (A OR B)          [Distributive]
      = 1 AND (A OR B)                     [Complement]
      = A OR B                             [Identity]

Final result: A OR B
```

Worked Example: Build a Logic Circuit from an Expression

To draw a logic circuit diagram for `(A AND B) OR (NOT C)`:



To draw a circuit: start at the inputs (right to left), draw each operation as a gate, connect with wires. The output of the AND gate and the NOT gate both feed into the OR gate, which produces Q.

> **Exam Tip:** For Boolean algebra simplification questions, **show every step** and name the law used at each step — these are method marks. The most useful laws to memorise for GCSE are: **Complement** (A AND NOT A = 0; A OR NOT A = 1), **Absorption** (A AND (A OR B) = A), **Idempotent** (A OR A = A), **Double negation** (NOT NOT A = A), and **De Morgan's**. When asked to draw a logic circuit: identify the operations in order (inner brackets first), draw gates for each, and connect them left-to-right from inputs to output. Always verify your simplified expression by plugging in test values.